

Типы данных в Python — что это такое?

Python — это один из самых популярных языков программирования. Его модель данных заключается в том, что всё считается объектом. Возьмём для примера `class A`, где `class` — это тип данных, выбранный разработчиком, и `A` — экземпляр этого класса. Здесь объект — как сам класс, так и его экземпляр.

Объект — это область памяти компьютера, он описывается с помощью значения и типа. Значение — это, собственно, данные, которые хранит объект. Оно присваивается с помощью оператора `(=)`. Тип — это классификация данных, определяющая значения переменной или объекта, а также какие операции можно с ними выполнить.

Возьмём для примера функцию `x = 1`. В данном случае `x` — это объект, а `1` — значение, оно определяет его тип — число.

Встроенные типы данных

Все типы

данных в Python делятся на встроенные и описанные. Первые — это стандартные структуры, которые по умолчанию включены в язык программирования. Вторые — это классы, их разработчик прописывает в коде самостоятельно по шаблонам.

Рассмотрим, какие типы встроенных данных используются в Python:

- Мутабельные — изменяемый тип данных, то есть их можно скорректировать после написания. Например, списки, словари и множества.
- Иммутабельные — неизменяемый тип данных. В Python в них нельзя внести изменения. Сюда можно отнести числа, строки, кортежи, `frozenset`, `float` и `bool`.

Новичкам рекомендуется использовать именно встроенные типы данных, потому что они упрощают процесс написания кода

Переменные

Переменные — это контейнеры для хранения значений, которые могут быть изменены во время выполнения программы. Это ключевое понятие любого языка программирования, без которого трудно представить разумное написание кода.

Python — язык с

динамической и при этом строгой типизацией. Динамическая типизация значит, что, допустим, объявив переменную `x = 1`, потом её значение можно поменять на `x = [a, b, c, d]`. То есть у переменной можно корректировать не только значение в рамках одного типа — допустим, единицу изменить на двойку, — но и менять тип данных с одного на другой.

При этом в Python переменная напрямую не содержит значение, а ссылается на него. Проще говоря, переменная — это просто название, которое ассоциируется со ссылкой на какое-то значение в области памяти. Именно поэтому смена типа данных переменной `x` — это, по сути, изменение указателя области памяти с одного места на другое.

Строгая типизация означает то, что нельзя совершать математические действия с разными типами данных. Например, в Python, в отличие от JavaScript, нет возможности сложить число `a = 1` и строку `b = "1"`. Такое выражение выдаст ошибку.

Числа (`int`, `float`, `complex`)

Это самый простой встроенный, но при этом неизменяемый тип данных.

Новичку это может показаться странным: ведь можно написать `x = 1`, а потом добавить `x += 1` (присвоить сумму текущего и заданного значения), и `x` изменится с 1 на 2. Но на самом деле в этом случае меняется не значение, а ссылка числа 1 на ссылку числа 2.

Вопрос «Какие типы данных в Python изменяемые?» могут задать начинающему программисту на собеседовании, и многие ошибаются, называя числа. Избежать таких ошибок, детально изучить язык и научиться писать код по фреймворку Django можно на курсе «Python-разработчик». Занятия проводят программисты Яндекса и других крупных компаний, а программа обновляется каждые три месяца.

Станьте Python-разработчиком с нуля

Освойте IT- профессию с нуля за 10 месяцев. Будете много практиковаться, получите востребованные навыки и реальный опыт.

13 проектов для портфолио;
помощь с поиском работы до 7 месяцев после выпуска;
приглашения на собеседования чаще на 38% и зарплата выше на 9%*

Подробнее о курсе

Попробовать бесплатную часть →

*Согласно исследованию Практикума, по сравнению с соискателями, не заканчивавшими никаких курсов

В Python есть три типа данных чисел:

1. `int` — целые числа вида 0, 1 или 10.
2. `float` — числа с плавающей запятой, имеющие дробную часть, например, 1,5 или 1,0.
3. `complex` — комплексные числа вида `a + bi`, где `a` и `b` — действительные числа, а `i` — мнимая единица.

Число может быть как положительным, так и отрицательным. В отличие от некоторых других языков, где переменная обычно не бывает длиннее 64 бит, в Python нет ограничения по размеру, есть возможность записать число любой длины. Правда, стоит учесть, что сложение очень длинных чисел может занимать много времени.

Кроме того, в этом типе данных есть полезная оптимизация, которая называется интернирование объектов. Оно заключается в том, что Python запоминает числа от -5 до 256, которые чаще всего используются в языке. Интернирование нужно для того, чтобы ускорить работу кода и каждый раз не выполнять сложение чисел.

Строки (`str`)

Строка — это набор символов, заключённых в кавычки. Одинарные и двойные кавычки записывают данные в одну строку, а тройные — в несколько. Такой тип данных в Python может содержать любые объекты, например, буквы, цифры или другие данные.

```
'Hello, username'
"""Hello,
username!"""
```

Строки — неизменяемый тип данных, но с ним можно проделывать некоторые манипуляции. Например, есть возможность математически сложить две строки или выполнить оператор `+=`. Однако в недрах Python при этом создаётся новая строка, то есть первоначальные данные не меняются, а создаются новые.

На этой особенности строк работодатели часто «ловят» джунов на собеседованиях. Например, предложить складывать строки через оператор `+=` при создании цикла неэффективно. Лучше сделать это через функцию `join`.

```
cities = ["Moscow", "Paris", "Rome"]
separator = ", "
result = separator.join(cities)
print(result) # Moscow, Paris, Rome
```

В Python встроена возможность записывать строки не только латиницей, но и в Unicode, который включает, например, русский и китайский. Это важно, поскольку буквы английского алфавита обычно занимают 1 байт, а русские могут содержать до 4 байт. Такая кодировка несколько усложняет работу языка, но удобна для разработчиков.

Списки (List)

Список — это изменяемый тип данных, который хранит в себе произвольные объекты. В него, например, можно включить как строку, так и число.

При этом в Python есть возможность сортировать элементы списка в любом нужном порядке.

```
my_list = ["Welcome", 1.34, True]
```

Этот тип данных можно расширять или сокращать. Удобнее всего добавлять и убирать значения с конца, особенно если список большой.

Подобные манипуляции с началом или серединой займут много времени, потому что Python ищет элементы линейным поиском. Например, при сокращении списка находит нужный объект, удаляет его и сдвигает на одну позицию влево остальные элементы. Это не очень эффективно, поэтому удаление элементов — это трудозатратная операция.

Однако в целом работать со списками в Python удобно благодаря тому, что он хранит внутри них не сами элементы, а ссылки на них, а сама ссылка занимает 8 байт. При добавлении новых элементов заполняется выделенная под них память, а когда свободное место заканчивается, его увеличивают с запасом. Это позволяет существенно ускорить расширение списка.

В Python есть простые списки, которые записываются в квадратные скобки. Также в них можно вложить другой список, например, так: `nested_list = [[1, 2], ["a", "b"]]`, или создать пустой, чтобы позже заполнить его данными: `empty_list = []`. Кроме того, существуют срезы списков — способ доступа к подмножеству объектов списка. Срез возвращает новый список, содержащий элементы исходного списка в заданном диапазоне индексов. Например, `slice_list = my_list[1:3]`, где `slice_list` содержит объекты "Hello" и 3.14.

Отдельно стоит выделить кортежи — неизменяемые списки, где объекты сортируются строго по порядку. Их выбирают для создания статического набора простых данных, который нет необходимости изменять в дальнейшем. Этот тип данных в Python удобен тем, что в нём можно быстро найти элемент по его номеру, потому что они идут по порядку.

При этом в произвольное место кортежа нельзя присвоить какое-то значение, как в других видах списков.

Кортежи заключаются в круглые скобки и разделяются запятыми: `named_tuple = ("apple", "banana", "cherry")`.

Множества (Set)

Множество — изменяемый тип данных, хранящий в себе значения, которые гарантированно не будут повторяться.

К примеру, если добавить в него 10 единиц, там всё равно останется только одна единица.

При этом, в отличие от списка, объекты во множестве хранятся в произвольном порядке.

```
this_set = {"B", "A", "C"}
```

При этом в Python есть отдельный вид множества `frozenset` — это неизменяемый тип данных. В отличие от обычного множества, значения `frozenset` записываются в двойных фигурных скобках.

```
frozen_set = frozenset({"B", "A", "C"})
```

Главное преимущество `frozenset` состоит в том, что для хранения объектов используется такой тип шифрования данных, как хеш-функция. Проще говоря, элемент записывается в виде множества чисел. Таким образом, бесконечному числу объектов можно сопоставить конечный набор значений хеш-таблицы. Вероятность, что у двух объектов будет одинаковый хеш, пренебрежительно низкая. Python понадобится пара операций, чтобы среди этих объектов отличить нужный, но происходит это быстро.

В целом множество удобно тем, что позволяет быстро проверять, есть ли в нём конкретный элемент. Допустим, чтобы проверить наличие объекта в списке, надо пройти каждый из них по порядку. А понять, что элемента там нет или их несколько, можно только пройдя весь список. Во множестве поиск объектов проходит гораздо быстрее, потому что каждый из них встречается только один раз.

Со множеством в Python можно проделывать разные манипуляции: объединять, вычитать, искать между ними пересечения. В них также есть возможность добавлять различные типы неизменяемых данных, например, в обычное множество можно включить `frozenset` или кортеж. Но нельзя положить изменяемые данные, например, списки или словари.

```
this_set = {"apple", "banana", frozenset(["cherry", "orange"]), (1, 2, 3)}
```

Словари (Dictionary)

Словарь — это более продвинутый вариант множества.

Он также хранит данные в виде хеша, но содержит не просто элементы, а пару «ключ-значение».

То есть для того, чтобы вывести значение, нужно ввести ключ.

```
d1 = {
    'doctor': 'Gregory House',
    'pilot': 'Anakin Skywalker'
    'wizard': 'Gandalf The White'
}
```

В словарях ключ — immutable тип данных, а значение может быть как изменяемым, так и неизменяемым. Если изменится объект, то изменится его хеш, а менять его динамически — значит пересобирать хеш-таблицу.

Поэтому в большинстве случаев требуется неизменяемый ключ, чтобы сохранить хеш.

В Python есть `defaultdict` — это подкласс встроенного словаря, который позволяет автоматически присваивать значение по умолчанию для ключей, которые ещё не были определены. Это особенно полезно при работе с данными, где некоторые ключи могут отсутствовать или быть неопределёнными. Создать `defaultdict` можно так:

```
from collections import defaultdict
```

```
# Создание defaultdict с типом данных по умолчанию
my_dict = defaultdict(list)
```

```
# Добавление элементов в словарь
my_dict["apple"].append("red")
my_dict["banana"].append("yellow")

print(my_dict) # выведет {'apple': ['red'], 'banana': ['yellow']}
```

В Python есть возможность объединять словари, создавая новые объекты. Также их можно использовать в качестве счётчика количества пар «ключ-значение» (counter). Запустить такой счётчик можно с помощью функции `len()`. Она возвращает длину (количество ключей) словаря. Пример использования: `print(len({"apple": 1, "banana": 2}))`.

Преобразование типов данных

В Python есть возможность менять подтипы данных с одного на другой.

Так, целое число можно преобразовать в `float` — число с дробью, или наоборот. Это пригодится, например, когда нужно отбросить дробную часть числа, если `x` был `float`, то `int(x)` — это `x`, округлённый вниз.

Также можно менять и тип данных в целом. Допустим, практически любой объект преобразуется в строку. А если нужно убрать все дубликаты в списке, достаточно преобразовать его во множество, и повторяющиеся объекты убираются автоматически.

```
values = [1, 2, 1, 1, 3, 2, 1, 3, 2]
print(set(values)) # выведет {1, 2, 3} в произвольном порядке
```

Для настройки корректной работы кода в Python пригодится возможность преобразования строк в числа. Допустим, ввод какого-то значения осуществляется через функцию `input`, а `input` принимает

от пользователя произвольную строку. При этом нужно ввести конкретное число от пользователя и прибавить к нему единицу. Если просто написать `x = input()` и добавить `x += 1`, то код выведет ошибку. Чтобы её избежать, нужно сначала преобразовать строку в число.

```
x = int(input())
x += 1 # ошибки не будет
```

```
x = input()
x += 1 # будет ошибка TypeError
```